

02 - Control Structures

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

Contents

1	Conditionals / 条件判断	2
1.1	Basic Logic / 基础逻辑	2
1.2	The Inline Choice / 行内选择	2
2	Functions / 函数基础	2
2.1	Anatomy of a Function / 函数构造	2
2.2	Inputs and Outputs / 输入与输出	3
3	Scope / 作用域的秘密	3
4	Loops / 循环: 程序如何做重复工作	3
4.1	For Loops / For 循环	3
4.2	While Loops / While 循环	4
4.3	Control Keywords / 循环控制关键字	4

1 Conditionals / 条件判断

1.1 Basic Logic / 基础逻辑

【Concept / 核心概念】1

If-statements / If 语句 If-statements allow you to execute parts of your code based on whether a condition is met [1]. As long as the statement after "if" is **True**, the **indented** code block will run [1].

If 语句允许程序根据条件是否满足（评估为 **True** 或 **False**）来决定执行哪段代码 [1]。只要条件为真，** 缩进 ** 的代码块就会运行 [1]。

【Example / 实战案例】1

Price Classification / 价格分级案例 课件中通过以下案例展示了 **elif** 和 **else** 的用法 [2, 3]:

```
price = 42
if price > 100:
    print("expensive")
elif price > 50:      # Check additional conditions / 检查额外条件
    print("moderate")
else:                 # Runs if code within if/elif did NOT run / 以上都不满足时执行
    print("cheap")
```

1.2 The Inline Choice / 行内选择

【Example / 实战案例】1

Ternary Operator / 三元运算符案例 课件展示了单行条件判断的写法 [4]:

```
name = "John"
# If length < 3, output "short", otherwise nothing is specified in this snippet
if len(name) < 3:
    print("your name is short!")
```

2 Functions / 函数基础

2.1 Anatomy of a Function / 函数构造

【Concept / 核心概念】1

Two Parts of a Function / 函数的两大部分 Every function has two parts [5]:

1. **Function Signature:** Defines the name and inputs (parameters) [5]. / **函数签名:** 定义名字和输入参数。
2. **Function Body:** The code that runs when the function is used [5]. / **函数体:** 实际执行的代码块。

2.2 Inputs and Outputs / 输入与输出

- **Argument:** A value passed to the function as input [6]. / **参数:** 作为输入传递给函数的值。
- **Return:** When a function is done, it returns a value as output [6]. / **返回值:** 执行完毕后作为结果输出。

【Warning / 避坑指南】1

Calling Errors / 调用错误 Functions like `len()` require exactly one argument [6]. Calling `len()` without input will raise a `TypeError` [7].

像 `len()` 这样的函数必须传入一个参数 [6]。不带参数调用会导致类型错误 (`TypeError`) [7]。

3 Scope / 作用域的秘密

【Concept / 核心概念】1

Local vs. Global / 局部与全局 Names are unique [8]. A function body has a **local scope**, while code outside is in the **global scope** [9].

变量名是唯一的 [8]。函数体内部拥有一个独立的“局部作用域”，而函数外部的代码则属于“全局作用域” [9]。

【Example / 实战案例】1

Scope Hierarchy / 作用域优先级案例 Python 查找变量时先看局部，找不到才去全局找 [9]。

```
x = 2                                # Global x
def func():
    x = 3                            # NEW local x, doesn't change global x!
    print("Inside:", x)

func()                               # Inside: 3 [4]
print("Outside:", x)                # Outside: 2 [4]
```

4 Loops / 循环：程序如何做重复工作

4.1 For Loops / For 循环

【Concept / 核心概念】1

The `range()` Function / Range 函数 `range(1, 11)` produces numbers from 1 up to (but NOT including) 11 [10].

`range(1, 11)` 产生 1 到 10 的序列，**不包含**终点 11 [10]。

【Example / 实战案例】1

Printing Numbers / 打印数字案例

```
for i in range(1, 11):  
    print(i) # Prints 1, 2, ..., 10 [11]
```

4.2 While Loops / While 循环

【Concept / 核心概念】1

Uncertain Tasks / 不确定次数的任务 Used when we only know **when to stop**, but not how many times to run [10].

用于只知道“什么时候停”，但不知道要运行几次的情况 [10]。

【Warning / 避坑指南】1

Infinite Loops / 无限循环风险 A **while** loop could run **forever** if the condition never becomes **False** [3].

如果条件永远不为假，**while** 循环会无休止运行 [3]。务必在循环内更新计数器：`counter = counter + 1` [12]。

4.3 Control Keywords / 循环控制关键字

- **break**: Aborts the loop entirely [10, 11]. / 直接终止整个循环。
- **continue**: Skips the rest of the current iteration [10, 11]. / 跳过本次循环剩余部分。
- **pass**: A placeholder that does nothing [10, 11]. / 占位符，什么都不做。